Registry: Docker Hub explorer

1. Product and API discovery

Registry supports a small developer workflow: search for an image, inspect its repository, find a tag, and copy a pull command. It has two responsive pages, explicit repository search and debounced tag search, shareable URL state, and ten curated popular repositories. The original assignment is preserved in the repository.

API investigation used the assignment's Docker Hub references, candidate endpoints and the official API reference. Small unauthenticated requests verified search, pagination, public details, tags, empty responses and missing repositories. Evidence is in docs/api-investigation.md.

Endpoints used (host: https://hub.docker.com):

- GET /v2/search/repositories/ with query, page and page_size.
- GET /v2/namespaces/{namespace}/repositories/{repository}.
- GET /v2/namespaces/{namespace}/repositories/{repository}/tags with page, page_size and optional name.

Details and tags routes are documented. Search and the tag-name filter were not found in the reference reviewed, so support is an observed integration dependency. Public tags worked anonymously despite authentication being shown in the tag reference. Search pages returned different results; name=alpine narrowed nginx tags across multiple pages. Deliberately nonexistent search terms returned zero matches; a missing repository returned HTTP 404.

Identity. Official nginx results had an empty owner and an unqualified name; details required namespace library. Nonofficial nginx/nginx-ingress also had an empty owner, so namespace comes from the qualified name. These differences are normalized in one adapter. The app does not guess ambiguous identities or assume every image has a latest tag.

Popular images and logos. The showcase selects nginx, Redis, PostgreSQL, Node.js, Python, MySQL, MongoDB, Ubuntu, Debian and Apache httpd. It is labeled as curated, not a measured global ranking. Showcase descriptions and timestamps come from Docker Hub, with a five-minute server cache. Ten Devicon logos are stored locally at a recorded source commit, with the MIT license and trademark attribution. Other results and failed images use a two-character name fallback with an accessible label. This avoids decorative API requests; full logo coverage remains a limitation.

Server cache. A small in-memory cache stores the ten showcase repositories' fetched details and an expiry timestamp for five minutes. Requests share any active fetch through the same Promise. After expiry, the next request refreshes the data; restarting the server clears it. This cache uses no external library or Redis and does not cache searches, individual repository details or tags.

Sizes. Multi-platform responses contain different sizes and unknown-platform entries. The UI lists each known platform and its size, including variants and Windows versions. Long lists expand within the row; missing sizes are unavailable. Pagination appears above and below both tag and repository search results. It does not sum architectures or substitute an unexplained full_size value. Binary units describe reported image bytes; cached layers and other platforms can change transfer size.

2. Implementation and product decisions

Toolkit overview. React enables reusable UI components; strict TypeScript makes shared contracts explicit. Vite supports development and production builds. React Router handles shareable URLs and navigation. TanStack Query (React Query) manages server state and caching, reducing component logic. Express keeps the API and static hosting lightweight. Zod validates upstream data at runtime. Plain CSS supports a custom responsive design. Vitest/Supertest test logic and API boundaries; Playwright/axe-core check browser behavior and accessibility rules.

Architecture. A client API module separates fetch from rendering; a Docker Hub adapter owns upstream access and normalization. Four read operations cover popular repositories, search, details and tags. Shared contracts contain only consumed fields. One package keeps this small application straightforward to run and maintain.

Why a server. A small integration layer keeps Docker Hub-specific behavior out of the UI. It validates upstream responses, normalizes them into shared contracts, centralizes timeouts and error handling, and shares showcase caching and rate-limit cooldown across clients in the same server process. This gives the frontend a focused API and keeps upstream changes isolated in the adapter. Missing CORS headers in tested responses were an additional practical reason. The server also serves the built UI and deep links, without requiring credentials or a database.

Request behavior. React Router stores search and page in URLs and preserves the return link. Repository search uses explicit submission to keep results stable while users compose a search, updating only when they choose to submit. Within a repository, tags behave like list filtering: a 500 ms debounce gives quick feedback while reducing requests during typing. Enter and the button still submit immediately. This is a UX preference, not a requirement; repository search could also use debounce. Automatic updates replace browser history and reset pagination. TanStack Query (React Query) separates state by query key, passes abort signals and deduplicates requests. Data stays fresh for one minute; unused queries are eligible for removal after five. Automatic retries, focus and reconnect refetching are disabled.

Failure boundaries. Details and tags load independently, with loading, empty, error and retry states. The server accepts known operations, bounds and validates input, encodes parameters, fixes page sizes, times out requests after ten seconds, and emits safe errors. It never follows arbitrary next-page URLs. A 429 starts a shared cooldown for subsequent upstream requests, capped at one hour; the affected view disables its retry button for the reported delay.

Useful additions. Server-side tag search and pagination help with large repositories. Visible commands make copying transparent and remain selectable if clipboard permission fails. Success appears only after writing and clears after 2.5 seconds. A selected read-only field supports manual copying if clipboard access fails. Clear controls preserve input focus; pagination returns focus to the results. Existing cards remain visible but inactive while another page of the same search loads, avoiding temporary layout collapse. Official-image labels, owners, UTC dates, platform labels, focus styles and mobile tag layout reduce ambiguity and support keyboard use.

Deliberate limits. No accounts, database, favorites, analytics, security scoring or deployment. The app does not fetch all tags, infer stability from latest, render upstream HTML descriptions, or claim its showcase is a live ranking. Full logo coverage and repository-wide platform filtering remain deferred.

3. Verification, lessons and next steps

Automated verification. Strict type checking, ESLint, unit/integration tests and production build passed. Deterministic tests cover identity, exact commands, size policy, missing data, validation, upstream failures, cooldown and caching. Chromium scenarios use the production UI with fixture APIs: ten cards, explicit search, pagination reset, direct links and refresh, return context, clipboard contents and failure, independent tag errors and recovery, concurrent searches, keyboard use, narrow layouts and automated axe accessibility scans. Fixture tests check these scenarios; they do not establish live API availability.

Live verification. A separate smoke test queried all ten showcase repositories, searched and paginated nginx, retrieved namespaced details, searched tags across pages, checked empty results and handled a missing repository. Browser checks against the local production server inspected real search results, official and namespaced details, tags and exact clipboard content. The verification log records final checks and limits.

Lessons learned. Real API responses need inspection and runtime validation, not just TypeScript types: missing owner fields showed why normalization belongs at the integration boundary. Data also needs a clear product meaning: a multi-platform tag has several sizes, so showing each platform's size is more informative than choosing or combining them silently. Finally, correct results alone do not guarantee a usable interaction. A rapid-search test exposed focus loss during URL updates, reinforcing the value of testing typing, focus and navigation as a continuous user flow.

Remaining risks. The application depends on Docker Hub APIs that may change outside our control. Browser testing covered Chromium, including keyboard and responsive checks; Firefox, Safari and a full screen-reader audit remain untested.

With more time. Add filtering by operating system and CPU architecture, and an image-based search flow with a dedicated image page where users can search its associated tags. Investigate whether the upstream API reliably supports these capabilities. Limited API support could create implementation challenges around grouping, performance and pagination, requiring an alternative approach.

AI assistance. I used AI to support API investigation, implementation, tests, visual review and documentation. Verification included compiler/lint tools, fixture tests, real API calls and browser interactions. I remain responsible for understanding the solution and the decisions behind it.

References: https://docs.docker.com/reference/api/hub/latest/ ; https://github.com/echo-ha/registry-challange ; https://github.com/devicons/devicon (pinned commit and license included).